



Chuletillo de Streams



◆ Operaciones intermedias ◆

(se pueden encadenar)

Son como “filtros” o “transformaciones”. Devuelven *otro stream*.

Método	Qué hace	Ejemplo con <code>items</code> (<code>List<Item></code>)
<code>filter</code>	Se queda solo con los que cumplen una condición	<code>items.stream().filter(i -> i.costo() > 500)</code>
<code>map</code>	Transforma cada elemento en otra cosa	<code>items.stream().map(i -> i.getDetalle())</code> → nombres
<code>mapToDouble</code> / <code>mapToInt</code>	Como <code>map</code> , pero a números primitivos	<code>items.stream().mapToDouble(Item::costo)</code>
<code>limit(n)</code>	Se queda con los primeros n elementos	<code>items.stream().limit(3)</code>
<code>sorted</code>	Ordena los elementos según un criterio	<code>items.stream().sorted((i1, i2) -> Double.compare(i1.costo(), i2.costo()))</code>

◆ Operaciones terminales ◆

(cierran el stream)

Dan un resultado concreto (número, lista, booleano...).

Método	Qué hace	Ejemplo
<code>count</code>	Cuenta cuántos hay	<code>(int) items.stream().count()</code>

sum	Suma números (cuando hiciste <code>mapToInt</code> o <code>mapToDouble</code>)	<code>items.stream().mapToDouble(Item::costo).sum()</code>
average	Calcula el promedio (devuelve <code>Optional</code> , usamos <code>.orElse(0)</code>)	<code>items.stream().mapToDouble(Item::costo).average().orElse(0)</code>
max / min	Devuelve el mayor o menor según un comparador	<code>items.stream().max((i1, i2) -> Double.compare(i1.costo(), i2.costo())).orElse(null)</code>
anyMatch	Devuelve <code>true</code> si <i>alguna</i> cumple la condición	<code>items.stream().anyMatch(i -> i.getCantidad() > 10)</code>
allMatch	Devuelve <code>true</code> si <i>todos</i> cumplen	<code>items.stream().allMatch(i -> i.getCostoUnitario() > 0)</code>
noneMatch	Devuelve <code>true</code> si <i>ninguna</i> cumple	<code>items.stream().noneMatch(i -> i.getDetalle().isEmpty())</code>
findFirst	Devuelve el primer elemento que cumple (o <code>null</code>)	<code>items.stream().filter(i -> i.getDetalle().startsWith("A")).findFirst().orElse(null)</code>
collect(Collectors.toList())	Vuelve a una lista	<code>items.stream().filter(i -> i.costo() > 500).collect(Collectors.toList())</code>

◆ Mini-guía mental ◆

1. Quiero filtrar cosas → `filter`
2. Quiero transformar cosas → `map`
3. Quiero números → `mapToInt/mapToDouble`
4. Quiero sumar o promediar → `sum`, `average`

5. Quiero saber si alguno/todos cumplen → `anyMatch` / `allMatch` / `noneMatch`
6. Quiero ordenar → `sorted`
7. Quiero máximo/mínimo → `max` / `min`
8. Quiero una lista al final → `collect(Collectors.toList())`